English version: docs/Docker-Synology-Guide.pdf

DulyInspect als Einzelinstanz

Einrichtung mit Docker und im Synology Container Manager

Diese Anleitung beschreibt, wie Sie DulyInspect komplett, also App und Server, als **einen** Container betreiben: auf einem Linux-Server mit Docker oder auf einer Synology DiskStation. Sie erklärt alle Umgebungsvariablen, den Datenordner, HTTPS, die Datensicherung und Updates.

Stand	3. November 2026 · App 2.90.0
Repository	github.com/tommx0/dulyinspect
Image	dulyinspect
Lizenz	GNU AGPL-3.0, freie Software (Datei LICENSE im Repository)
Status	Vorabversion: auf Intel/AMD (amd64, GitHub-Workflow) und ARM (arm64) automatisch getestet; Safari/iOS und echte Synology-Geräte noch offen (Abschnitt 10)

Inhalt

1. Überblick
2. Voraussetzungen
3. Image bauen und bereitstellen
4. Umgebungsvariablen
5. Datenordner und Freigaben
6. Betrieb mit Docker (Linux-Server)
7. Synology Container Manager
8. Datensicherung
9. Updates
10. Bekannte Einschränkungen und Fehlermeldungen

1. Überblick

Das Image enthält alles, was DulyInspect braucht: die App (Browser-Oberfläche), die Datenbank (PostgreSQL), Anmeldung, Dateiablage, Echtzeit-Abgleich und die Server-Funktionen. Nach außen gibt es nur **einen Port (8080)** und **einen Datenordner (`/data`)**.

- **Eine Instanz je Container:** ein Unternehmen, eine Datenbank, ein Admin-Bereich. Mandanten wie im Coolify-Stack gibt es hier nicht.
- **Der erste Admin** wird beim ersten Aufruf im Browser angelegt, geschützt durch einen Einrichtungsschlüssel aus dem Datenordner (für automatische Installationen wahlweise aus Umgebungsvariablen).
- **Updates** ersetzen nur den Container. Alle Daten bleiben im Datenordner, und vor jedem Versionswechsel sichert der Container die Datenbank automatisch.
- **Lizenz:** freie Software unter der GNU AGPL-3.0. Nutzung, auch in Firmen, ist kostenlos. Wer das Programm verändert weitergibt oder als Dienst anbietet, muss seinen Quellcode offenlegen. In der App führt „Lizenz“ unten auf jeder Seite zum Lizenztext.
- **HTTPS** macht ein Reverse Proxy davor, auf der Synology der eingebaute. Ohne HTTPS funktionieren Kamera (QR-Scan) und die Installation als App auf dem Handy nicht.

2. Voraussetzungen

Was	Anforderung
Prozessor	Intel/AMD (`x86_64` , Image für `linux/amd64`) oder ARM 64 Bit (`aarch64` , Image für `linux/arm64`). Das Image muss zur Plattform passen. Prüfen per SSH mit `uname -m` , auf der Synology auch unter **Info-Center → Allgemein**.
Arbeitsspeicher	Gemessen rund 500–600 MB im Betrieb. Empfohlen: mindestens 1,5 GB frei, also ein Gerät mit 2 GB RAM oder mehr (besser 4 GB).

Speicherplatz	Image rund 1,9 GB; die Datei zum Übertragen hat rund 0,35 GB (Download von GitHub als ZIP, die entpackte `.tar` rund 1,2 GB), dazu die Daten: Datenbank anfangs unter 100 MB, Fotos und Protokolle je nach Nutzung. Dazu die Sicherungen.
Docker	Linux: Docker Engine ab Version 24 mit dem Compose-Plugin. Synology: DSM 7.2 oder neuer mit dem Paket **Container Manager**.
Internet	Beim **ersten Start** und beim ersten Aufruf der Benutzerverwaltung lädt der Container Programmbibliotheken nach. Er braucht dann ausgehenden Internetzugang (Abschnitt 10).
Adresse und Zertifikat	Ein Name wie `pruefung.firma.de`, der auf den Server zeigt, und ein Zertifikat (Let's Encrypt), für die Nutzung im Haus und unterwegs.

3. Image bauen und bereitstellen

Das Image wird als Datei (`.tar`) weitergegeben. Es gibt zwei Wege, sie zu erzeugen.

3.1 Über GitHub (empfohlen für Intel/AMD)

1. Im Repository `tommx0/dulyinspect` auf GitHub: **Actions** → **Build image (amd64)** → **Run workflow**.
2. Der Lauf dauert etwa 10–15 Minuten. Er baut das Image, startet es zur Probe und prüft dabei Erststart, Anmeldung, Dateiablage, Echtzeit-Verbindung, Sicherung, Update und Datenerhalt. Nur wenn alles besteht, entsteht die Datei.
3. Auf der Seite des Laufs unter **Artifacts** `dulyinspect-<Version>-amd64` herunterladen. Das ist eine ZIP-Datei. Entpacken ergibt die `.tar`-Datei für den Import (Abschnitt 6 bzw. 7.3). Die Datei steht 5 Tage zum Download bereit.

Mit „Also publish to ghcr.io“ landet das Image außerdem in der privaten GitHub-Registry. Das belegt dauerhaft Speicher des GitHub-Kontos (rund 0,35 GB je Version).

3.2 Lokal mit build.sh

Das Skript `build.sh` baut das Image aus dem App-Quellcode im Ordner `app/` des Repositorys und speichert es als `.tar`-Datei.

```
git clone https://github.com/tommx0/dulyinspect.git
cd dulyinspect
./build.sh linux/arm64      # für ARM
./build.sh linux/amd64      # für Intel/AMD (auf einem Mac mit Apple-Chip siehe unten)
test/smoke-test.sh dulyinspect:latest   # optional: dieselben Prüfungen wie auf GitHub
```

Ergebnis: `out/dulyinspect-<Version>-<Plattform>.tar` mit den Namen `dulyinspect:<Version>` und `dulyinspect:latest`. Welche App-Version enthalten ist, steht in `app/SOURCE`.

> **Bauen auf einem Mac mit Apple-Chip für Intel/AMD:** Dafür muss Docker Desktop x86-Programme ausführen können: **Docker Desktop** → **Settings** → **General** → **„Use Rosetta for x86_64/amd64 emulation on Apple Silicon“** einschalten und Docker neu starten. Ohne diese Einstellung bricht der Bau mit `exec format error` ab.

Auf einem Linux-Server mit Docker kann das Image auch direkt dort gebaut werden (gleiche Befehle, Plattform passend wählen). Dann entfällt die Datei.

4. Umgebungsvariablen

Variable	Pflicht	Bedeutung
`APP_URL`	ja	Die Adresse, unter der die App im Browser aufgerufen wird, genau so, mit `https://` und ohne Pfad, z. B. `https://pruefung.firma.de`. Daraus werden die Verbindung der App zum Server, die Anmeldung und die QR-Codes auf Etiketten und Protokollen gebildet. Vorgabe: `http://localhost:8080` (nur für Tests).
`ADMIN_EMAIL`	nein	Anmeldename (E-Mail-Adresse) des ersten Admins. Vorgabe: `admin@lokal`.
`ADMIN_PASSWORD`	nein	Nur für automatische Installationen: legt den ersten Admin schon beim Start an, mindestens 10 Zeichen. Ohne diese Variable legen Sie den Admin im Browser an (Abschnitt 7.7).
`ADMIN_NAME`	nein	Anzeigename des Admins. Vorgabe: `Administrator`.
`ADMIN_CHANGE_PASSWORD`	nein	`yes` (Vorgabe): Bei der ersten Anmeldung muss der Admin ein eigenes Passwort wählen. `no`: Das Startpasswort bleibt gültig.
`ADMIN_TWO_FACTOR`	nein	`yes`: Der Admin muss bei der ersten Anmeldung eine Authenticator-App einrichten. Vorgabe: `no` (später in der App unter „Benutzer“ einschaltbar).
`SETUP_WITHOUT_KEY`	nein	`yes`: Die Ersteinrichtung im Browser fragt den Einrichtungsschlüssel nicht ab. **Nur für Tests** auf dem eigenen Rechner: Wer die Seite zuerst aufruft, wird Admin.
`MAX_ACTIVE_USERS`	nein	Höchstzahl gleichzeitig aktiver Benutzer (laut Lizenz). Vorgabe: `2`.

Die früheren deutschen Namen (`APP_ADRESSE`, `ADMIN_PASSWORT`, `ADMIN_PASSWORT_AENDERN`, `ADMIN_ZWEI_FAKTOR`, `MAX_AKTIVE_BENUTZER`, `EINRICHTUNG_OHNE_SCHLUESSEL`) und die Werte `ja` / `nein` gelten weiter als Alias.

> **Der erste Admin wird genau einmal angelegt:** beim allerersten Start, solange es noch keinen Zugang gibt. Danach haben die `ADMIN_*`-Variablen keine Wirkung mehr. Ein späteres Ändern setzt also kein Passwort zurück. Entfernen Sie `ADMIN_PASSWORD` nach dem ersten Start aus der Konfiguration, denn Container-Einstellungen sind für alle Admins des Servers bzw. der Synology lesbar.

Der Normalfall: Ohne `ADMIN_PASSWORD` legt der Container keinen Benutzer an. Beim ersten Aufruf zeigt die App das Formular **Ersteinrichtung**: Dort legen Sie den ersten Admin an und drucken das Schlüsselblatt der Verschlüsselung. Damit niemand Fremdes Ihnen zuvorkommt, verlangt das Formular den **Einrichtungsschlüssel**. Er steht in der Datei `SETUP_KEY.txt` im Datenordner (auf der Synology in der File Station unter `docker/dulyinspect/data`) und im Protokoll des Containers. Sobald der Admin angelegt ist, löscht der Container die Datei. Danach führt die **Inbetriebnahme** der App durch Firmendaten, zu prüfende Anlagen, Checklisten, Impressum und Datenschutz, Erscheinungsbild, Etiketten und Benutzer.

Adresse ändern: `APP_URL` anpassen und den Container neu erzeugen. Die App übernimmt die neue Adresse beim nächsten Laden. Bereits gedruckte QR-Etiketten enthalten aber weiter die alte Adresse. Die Adresse sollte daher vor dem ersten Etikettendruck feststehen.

Alle übrigen Werte (Schlüssel, Datenbank-Passwörter) erzeugt der Container beim ersten Start selbst und legt sie im Datenordner ab. Sie werden nicht eingestellt.

5. Datenordner und Freigaben

Alles, was bleiben muss, liegt im Container unter `/data`. Dieser Pfad wird auf einen Ordner des Servers bzw. der Synology abgebildet (im Folgenden `data`). Der Container legt die Unterordner beim ersten Start selbst an und setzt die Rechte. Ein vorhandener Datenordner mit den früheren deutschen Namen (`geheim/`, `sicherung/` usw.) wird beim nächsten Start automatisch umbenannt.

Unterordner	Inhalt
`postgres/`	Datenbank. Nie im laufenden Betrieb kopieren oder öffnen (Abschnitt 8).
`storage/`	Fotos, Unterschriften, Prüfprotokolle (PDF), Nachweise, Logo.
`storage-xattr/`	Angaben zu den Dateien (Typ, Cache). Nur belegt, wenn der Ordner keine erweiterten Attribute kann, etwa bei Docker Desktop oder einer SMB/NFS-Freigabe. Gehört zu den Daten, nicht löschen.
`secrets/`	Schlüssel und Passwörter der Instanz (`instance.env`). **Nie löschen:** Ohne sie sind alle Anmeldungen ungültig, und verschlüsselte Einträge der Datenbank sind nicht mehr lesbar. Gehört in jede Sicherung und muss vertraulich bleiben.
`backups/`	Datenbank-Sicherungen: automatisch vor jedem Versionswechsel (`pre-*.sql.gz`, die letzten 5) und die geplanten (`live-*.sql.gz`, Abschnitt 8).
`caddy/`, `version`, `.lock`	Interne Verwaltung (Webserver, zuletzt eingerichtete Version, Sperre).

> **Ein Datenordner gehört zu genau einem Container.** Ein zweiter Container auf demselben Ordner würde die Datenbank beschädigen. Der Container verhindert das selbst: Er beendet sich mit `ERROR: /data is already in use by another container. Stop the other one first.` Bei einem Update daher immer erst den alten Container stoppen.

Die Dateien gehören im Ordner verschiedenen internen Benutzern des Containers. In der Dateiansicht des Servers erscheinen sie deshalb mit ungewohnten Besitzern oder als nicht lesbar. Das ist richtig so. Besitzer und Rechte nicht von außen ändern.

6. Betrieb mit Docker (Linux-Server)

> **Docker Desktop (Mac, Windows):** statt eines Ordners vom Rechner ein Docker-Volume verwenden (`dulyinspect-data:/data` und ein Abschnitt `volumes:`, siehe README). Docker Desktop kann auf Ordnern des Rechners keine Linux-Eigentümer speichern, Postgres startet dann nicht (`ERROR: /data cannot keep file owners …`). Linux-Server und Synology-Freigaben betrifft das nicht.

1. Ordner anlegen, z. B. `/opt/dulyinspect`, und die Datei `docker-compose.yml` aus dem Repository hineinkopieren.
2. Image laden (entfällt, wenn auf dem Server gebaut):

   ```
   docker load -i dulyinspect-2.90.0-amd64.tar
   ```

3. In `docker-compose.yml` die Umgebungsvariablen eintragen (Abschnitt 4), vor allem `APP_URL`.
4. Starten und das Protokoll verfolgen:

   ```
   cd /opt/dulyinspect
   docker compose up -d
   docker compose logs -f
   ```

   Der Container ist bereit, sobald im Protokoll `Setup complete, app 2.90.0 at https://….` steht. Beim ersten Start dauert das etwa 10–30 Sekunden.
5. Einen Reverse Proxy mit HTTPS davorsetzen (Caddy, nginx, Traefik …), der auf `http://127.0.0.1:8090` weiterleitet. Er muss **WebSocket** durchreichen (Header `Upgrade` und `Connection`), sonst funktioniert der Echtzeit-Abgleich zwischen mehreren Prüfern nicht. Port 8090 sollte nur lokal erreichbar sein, also `"127.0.0.1:8090:8080"` in `ports`, wenn der Proxy auf demselben Server läuft.
6. Im Browser die Adresse öffnen und den ersten Admin anlegen (Abschnitt 7.7). Den Einrichtungsschlüssel zeigt `cat data/SETUP_KEY.txt`.

Die Compose-Datei aus dem Repository:

```
services:
  dulyinspect:
    image: dulyinspect:latest
    container_name: dulyinspect
    restart: unless-stopped
    ports:
      - "8090:8080"            # links: Port auf dem Server/NAS, rechts immer 8080
    volumes:
      - ./data:/data           # alle Daten; bei Updates nie löschen
    environment:
      APP_URL: "https://pruefung.firma.de"   # Adresse, unter der die App im Browser
aufgerufen wird
      MAX_ACTIVE_USERS: "2"
      # Den ersten Admin legen Sie beim ersten Aufruf im Browser an (Einrichtungsschlüssel:
data/SETUP_KEY.txt).
      # Nur für automatische Installationen: Admin beim Start anlegen (nur beim allerersten
Start)
      # ADMIN_EMAIL: "admin@lokal"
      # ADMIN_NAME: "Administrator"
      # ADMIN_PASSWORD: "mindestens-10-Zeichen"
    stop_grace_period: 60s
```

Ohne Compose geht dasselbe mit einem Befehl:

```
docker run -d --name dulyinspect --restart unless-stopped --stop-timeout 60 \
  -p 8090:8080 -v /opt/dulyinspect/data:/data \
  -e APP_URL=https://pruefung.firma.de \
  dulyinspect:latest
```

Nützliche Befehle: `docker compose stop` (sauber anhalten), `docker compose start`, `docker compose logs --tail 100`. Den Container nicht mit `docker kill` oder `docker rm -f` beenden. Er übersteht das zwar, braucht dann aber beim nächsten Start eine Reparatur der Datenbank.

7. Synology Container Manager

Die Bezeichnungen entsprechen DSM 7.2 auf Deutsch. In anderen Versionen können Menüpunkte leicht anders heißen.

7.1 Container Manager installieren und Plattform prüfen

1. **Paket-Zentrum** → **Container Manager** installieren. Dabei entsteht der freigegebene Ordner `docker`.
2. **Info-Center** → **Allgemein**: Prozessor ablesen. Intel, AMD oder Celeron bedeutet Image `amd64`, ein ARM-Prozessor (z. B. Realtek RTD1619B) bedeutet `arm64`.

7.2 Freigabe und Ordner einrichten

1. In der **File Station** im Ordner `docker` den Unterordner `dulyinspect` anlegen und darin den Ordner `data`. Ergebnis: `/docker/dulyinspect/data` (auf dem Volume: `/volume1/docker/dulyinspect/data`).
2. **Systemsteuerung** → **Freigegebener Ordner** → **docker** → **Bearbeiten** → **Berechtigungen**: Zugriff nur für Administratoren. Der Ordner enthält mit `data/secrets` die Schlüssel der Instanz, und die Compose-Datei kann das Startpasswort enthalten.
3. Die Freigabe `docker` **nicht** per SMB oder AFP für normale Benutzer freigeben und im Ordner `data` nichts von Hand ändern.
4. Für die Sicherung (Abschnitt 8) empfiehlt sich ein Btrfs-Volume. Dann sind auch Snapshots der Freigabe möglich.

7.3 Image importieren

1. Die Datei `dulyinspect-<Version>-amd64.tar` (bzw. `-arm64`) in die Freigabe `docker` kopieren, z. B. mit der File Station (Hochladen).
2. **Container Manager** → **Image** → **Hinzufügen** → **Aus Datei hinzufügen** und die Datei wählen. Der Import dauert einige Minuten.
3. Danach erscheint `dulyinspect` mit den Tags `latest` und der Versionsnummer in der Liste.

Alternativ per SSH (**Systemsteuerung** → **Terminal & SNMP** → **SSH-Dienst aktivieren**): `sudo docker load -i /volume1/docker/dulyinspect-2.90.0-amd64.tar`

7.4 Projekt anlegen (empfohlen)

Ein Projekt ist eine Compose-Datei, die der Container Manager verwaltet. So stehen alle Einstellungen an einer Stelle, und Updates sind einfacher.

1. **Container Manager** → **Projekt** → **Erstellen**.
2. **Projektname:** `dulyinspect` · **Pfad:** `/docker/dulyinspect` · **Quelle:** „docker-compose.yml erstellen“.
3. Den Inhalt der Compose-Datei aus Abschnitt 6 einfügen und die Werte anpassen:
   - `APP_URL`: die spätere HTTPS-Adresse (Abschnitt 7.6), z. B. `https://pruefung.firma.de`.

- Port links von `:8080`: ein freier Port der Synology, z. B. `8090`. Die Ports 5000/5001 belegt DSM selbst.
- `./data:/data` bleibt so: `./` bezieht sich auf den Projektordner `/docker/dulyinspect`.

4. Die Seite zu den Webportal-Einstellungen (Web Station) überspringen, dort nichts einrichten.
5. **Fertig** mit „Projekt nach der Erstellung starten“. Der Container Manager erstellt und startet den Container.
6. **Container** → **dulyinspect** → **Protokoll**: warten auf `Setup complete, app 2.90.0 at ….`

7.5 Alternative: Container ohne Projekt

Wer lieber klickt: **Image** → **dulyinspect:latest** → **Ausführen**.

- **Allgemeine Einstellungen:** Containername `dulyinspect`, „Automatischen Neustart aktivieren“ einschalten.
- **Port-Einstellungen:** lokaler Port `8090` → Container-Port `8080`, TCP.
- **Volume-Einstellungen:** „Ordner hinzufügen“ → `docker/dulyinspect/data` → Mount-Pfad `/data`, nicht schreibgeschützt.
- **Umgebung:** die Variablen aus Abschnitt 4 einzeln hinzufügen.
- **Netzwerk:** `bridge` (Vorgabe).

7.6 HTTPS über den Reverse Proxy der Synology

1. Den Namen (z. B. `pruefung.firma.de`) im DNS auf die Synology zeigen lassen. Für den Zugriff von außen leitet der Router Port 443 an die Synology weiter. Nur im Haus genügt ein Eintrag im internen DNS.
2. **Systemsteuerung** → **Anmeldeportal** → **Erweitert** → **Reverse Proxy** → **Erstellen**:

	Quelle	Ziel
Protokoll	HTTPS	HTTP
Hostname	`pruefung.firma.de`	`localhost`
Port	443	8090

   Unter Quelle „HSTS aktivieren“ einschalten.
3. Im selben Eintrag: **Benutzerdefinierte Kopfzeile** → **Erstellen** → **WebSocket**. Das legt `Upgrade` und `Connection` an. **Ohne diesen Schritt** sehen Prüfer die Änderungen der anderen nicht sofort, sondern erst beim nächsten Laden.
4. **Systemsteuerung** → **Sicherheit** → **Zertifikat** → **Hinzufügen** → „Ein neues Zertifikat hinzufügen“ → „Zertifikat von Let's Encrypt abrufen“ für `pruefung.firma.de`. Danach unter **Einstellungen** das Zertifikat dem Reverse-Proxy-Eintrag zuweisen. Let's Encrypt braucht dafür Port 80 von außen.
5. Port 8090 nicht ins Internet freigeben, also im Router nicht weiterleiten. Mit der Synology-Firewall lässt er sich zusätzlich auf `localhost` beschränken.

7.7 Ersteinrichtung und Inbetriebnahme

1. In der File Station `docker/dulyinspect/data/SETUP_KEY.txt` öffnen und den Schlüssel kopieren. Er steht auch im Protokoll: **Container Manager** → **Container** → **dulyinspect** → **Protokoll**, in der Zeile

`Initial setup in the browser at …, setup key: XXXX-XXXX-…`.

2. `https://pruefung.firma.de` öffnen. Im Formular **Ersteinrichtung** den Schlüssel, Ihren Namen, Ihre E-Mail-Adresse (künftiger Anmeldename) und ein Passwort eintragen.
3. Das **Schlüsselblatt** der Verschlüsselung herunterladen, ausdrucken und sicher aufbewahren: Ohne es sind verschlüsselte Daten nicht wiederherstellbar.
4. Danach führt die **Inbetriebnahme** Schritt für Schritt durch die Einrichtung. Jeder Schritt lässt sich überspringen und später unter **Einstellungen** → **Inbetriebnahme** nachholen.
5. Auf dem Handy die Adresse öffnen und über „Zum Home-Bildschirm“ installieren. Erst dann funktioniert die Offline-Nutzung zuverlässig, besonders auf dem iPhone.

8. Datensicherung

Die Datenbankdateien unter `data/postgres` dürfen **nicht** im laufenden Betrieb kopiert werden. Eine solche Kopie kann unbrauchbar sein. Sicher sind diese drei Wege, am besten kombiniert:

8.1 Tägliche Datenbank-Sicherung (Aufgabenplaner)

Der Container bringt ein Sicherungsskript mit. Es schreibt im laufenden Betrieb eine vollständige, in sich stimmige Sicherung der Datenbank nach `data/backups/live-<Zeit>.sql.gz` und behält die letzten 14 (anpassbar).

1. **Systemsteuerung** → **Aufgabenplaner** → **Erstellen** → **Geplante Aufgabe** → **Benutzerdefiniertes Skript**.
2. **Allgemein:** Aufgabe „DulyInspect sichern“, Benutzer `root`.
3. **Zeitplan:** täglich, z. B. 02:30.
4. **Aufgabeneinstellungen** → **Benutzerdefiniertes Skript:**

   ```
   docker exec dulyinspect /usr/local/lib/instance/backup.sh 14
   ```

   Optional „Ausführungsdetails per E-Mail senden“ und „nur bei Fehler“, dann meldet die Synology fehlgeschlagene Sicherungen.

Auf einem Linux-Server dasselbe per Cron, z. B. `30 2 * * * docker exec dulyinspect /usr/local/lib/instance/backup.sh 14`.

8.2 Sicherung auf ein anderes Gerät (Hyper Backup)

Mit **Hyper Backup** (oder `rsync` / `restic` auf Linux) regelmäßig, z. B. täglich nach der Datenbank-Sicherung, diese Ordner auf ein anderes Gerät oder in die Cloud sichern:

- `docker/dulyinspect/data/backups`: die Datenbank-Sicherungen
- `docker/dulyinspect/data/storage`: Fotos, Protokolle, Nachweise
- `docker/dulyinspect/data/secrets`: Schlüssel der Instanz
- `docker/dulyinspect/docker-compose.yml`: Konfiguration

Die Sicherungen enthalten personenbezogene Daten und die Schlüssel der Instanz. Das Sicherungsziel deshalb verschlüsseln (in Hyper Backup: „Clientseitige Verschlüsselung“).

8.3 Snapshots (Btrfs)

Mit **Snapshot Replication** lässt sich die Freigabe `docker` z. B. stündlich als Snapshot festhalten. Ein Snapshot erfasst den Datenordner in einem Augenblick. Die Datenbank verhält sich beim Zurückspielen wie nach einem Stromausfall und repariert sich beim Start selbst. Snapshots sind praktisch, um schnell zurückzugehen, ersetzen aber keine Sicherung auf ein anderes Gerät.

8.4 Wiederherstellen

1. Container stoppen (**Projekt** → **Aktion** → **Stoppen** bzw. `docker compose stop`).
2. Den kompletten Ordner `data` aus einem Snapshot oder einer Sicherung zurückspielen, die bei **gestopptem** Container oder als Snapshot entstanden ist.
3. Container starten. Er prüft Datenbankstand und Version selbst.

> Die Wiederherstellung einer Instanz aus den `.sql.gz`-Dateien allein (z. B. auf neuer Hardware ohne Snapshot) ist noch nicht als fertiger Ablauf beschrieben und noch nicht getestet. Bis dahin: Snapshots bzw. eine Kopie des ganzen Datenordners bei gestopptem Container als Grundlage behalten, die Datenbank-Sicherungen zusätzlich.

9. Updates

Ein Update ersetzt nur das Image. Beim ersten Start mit einer neuen App-Version läuft automatisch dies ab:

1. Sicherung der Datenbank nach `data/backups/pre-<Version>-<Zeit>.sql.gz`. Schlägt sie fehl, startet der Container nicht.
2. Aktualisierung der Datenbank auf die neue Version (je Schritt eine Transaktion; bei einem Fehler bleibt der vorherige Stand).
3. Start der App mit der neuen Version. Die Geräte holen sie ohne Nachfrage: sofort beim nächsten Öffnen der App, sonst beim nächsten Seitenwechsel oder wenn die App in den Hintergrund geht. **Nie während einer Prüfung:** Solange auf einem Gerät eine Prüfung offen ist oder eine Prüfseite angezeigt wird, bleibt die alte Version, bis die Prüfung abgeschlossen ist.

Synology

1. Neue Image-Datei bauen (Abschnitt 3) und importieren (Abschnitt 7.3). Sie trägt wieder den Tag `latest`.
2. **Projekt** → **dulyinspect** → **Aktion** → **Stoppen**, dann **Aktion** → **Erstellen**. Der Container wird aus dem neuen Image neu erzeugt. Alternativ per SSH: `cd /volume1/docker/dulyinspect && sudo docker compose up -d --force-recreate`.
3. Im Protokoll auf `Update 2.86.0 → …: backup to …` und `Setup complete` achten.

Docker

```
docker load -i dulyinspect-<neue Version>-amd64.tar   # oder neu bauen
cd /opt/dulyinspect
docker compose up -d          # erkennt das neue Image und erzeugt den Container neu
docker compose logs -f
```

Ein **Zurück auf eine ältere Version** verhindert der Container, sobald die Datenbank schon aktualisiert ist (`… (older image?)`). Dann das Image der neuen Version wieder verwenden oder die Sicherung von vor dem Update zurückspielen (Abschnitt 8.4).

Alte Images belegen je rund 2 GB. Unter **Container Manager** → **Image** lassen sich nicht mehr verwendete löschen (bzw. `docker image prune`). Die vorherige Version zur Sicherheit einige Tage behalten.

10. Bekannte Einschränkungen und Fehlermeldungen

Stand dieser Vorabversion

- **Getestet** ist der Container automatisch auf Intel/AMD (amd64, bei jedem Bau über GitHub) und ARM (arm64): Erststart, Anmeldung, Fotos/Dateien, Echtzeit-Verbindung, Sicherung, Update mit Sicherung, Datenerhalt, Sperren. **Noch nicht** auf einem echten Synology-Gerät: Die Menüpunkte in Abschnitt 7 folgen DSM 7.2, sind aber nicht an einem Gerät nachgeklickt.
- **Safari/iPhone:** Der Echtzeit-Abgleich ist unter Safari noch nicht geprüft, Chrome ist geprüft.
- **Kein Demomodus, keine Vorschauversion:** Das Image ist für den Echtbetrieb gebaut. Es gibt keinen Hinweis und kein Wasserzeichen „Vorschauversion“, und der Demomodus ist fest aus. Neue Versionen laden ohne Rückfrage (Abschnitt 9).
- **Internetzugang:** Die Server-Funktionen laden beim ersten Aufruf Bibliotheken aus dem Internet (npm). Ohne ausgehenden Zugang scheitern der erste Admin aus den Variablen und die Benutzerverwaltung.
- **Postgres-Hauptversion:** Wechselt ein künftiges Image die Hauptversion der Datenbank (z. B. 15 auf 17), startet der Container mit alten Daten nicht (`ERROR: The data was created by Postgres …`). Für diesen Fall folgt eine eigene Anleitung, die Daten bleiben unverändert.

Meldungen im Protokoll

Meldung	Bedeutung und Abhilfe
`ERROR: /data is already in use by another container. Stop the other one first.`	Ein zweiter Container nutzt denselben Datenordner. Den anderen stoppen, dann erneut starten.
`ERROR: The database has schema …, this image only knows up to … (older image?)`	Das Image ist älter als die Daten. Das neuere Image verwenden.
`ERROR: The data was created by Postgres …, this image has Postgres …`	Wechsel der Datenbank-Hauptversion, siehe oben. Nichts wurde verändert.
`Backup failed – start aborted.`	Meist zu wenig Speicherplatz. Platz schaffen und erneut starten.
`ERROR in migration … – data unchanged, backup in /data/backups.`	Aktualisierung der Datenbank gescheitert, der vorherige Stand bleibt. Protokoll an den Support geben.
`ERROR creating the admin: …`	Meist ein zu kurzes Passwort (unter 10 Zeichen) oder fehlender Internetzugang. Variable korrigieren und neu starten.

`Initial setup in the browser at …, setup key: XXXX-XXXX-… (also in data/SETUP_KEY.txt)`	Kein Fehler: Den ersten Admin im Browser mit diesem Einrichtungsschlüssel anlegen (Abschnitt 7.7).
`First admin … created.`	Kein Fehler: Der erste Admin wurde aus `ADMIN_PASSWORD` angelegt. Die Variable jetzt aus der Konfiguration entfernen.

Typische Probleme

Problem	Ursache und Abhilfe
Kamera bzw. QR-Scan geht nicht, App lässt sich nicht installieren	Die App läuft ohne HTTPS. Über den Reverse Proxy mit Zertifikat aufrufen (Abschnitt 7.6).
Änderungen anderer Prüfer erscheinen erst nach dem Neuladen	WebSocket-Kopfzeilen im Reverse Proxy fehlen (Abschnitt 7.6, Schritt 3).
Anmeldung klappt, danach Fehler „keine Verbindung“	`APP_URL` stimmt nicht mit der Adresse im Browser überein. Anpassen und Container neu erzeugen.
Container startet nicht, Port belegt	Den linken Port (8090) auf einen freien ändern, auch im Reverse Proxy.